



DIPARTIMENTO DI **I**NGEGNERIA **E**LETTRICA E **T**ECNOLOGIE DELL'**I**NFORMAZIONE
SCUOLA POLITECNICA E DELLE SCIENZE DI BASE

LAUREA IN INGEGNERIA DELL'AUTOMAZIONE

Hierarchical VLMs Planning and Control with Ambiguity resolution

Candidato:
Vincenzo Palomba

Relatori:
Prof. Alberto Finzi

Academic Year 2025/2026

Outline

1. Deliberative Layer
2. Executive Layer
3. Control Layer
4. Setup and Experimental Results

Neuro-Symbolic Planning architecture

VLM Architectures

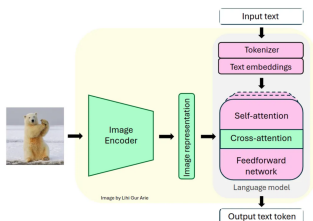


Figure: llama3 VLM architecture

Image + Text $\longrightarrow$ VLM $\longrightarrow$ Text Output

Language Encoder (Transformer)

- ▶ Encoders transform sequences into numerical representation called **embeddings**.
- ▶ A **self-attention mechanism** allows focus on important tokens.
- ▶ **Decoders** output the most statistically probable output.

Vision Encoder (CNN / ViT)

- ▶ Typically utilizes Convolutional Neural Networks (CNNs) or Visual Transformers (ViTs).
- ▶ A ViT processes an image into patches and treats them as sequences, akin to tokens.

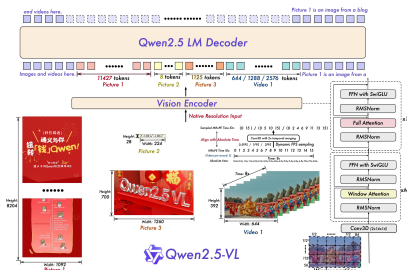
VLM Training methodologies

It is the mechanism that allow fusing information from both vision and language encoders so to correlate images with text and make decision on the 2 modalities together:

- ▶ Contrastive learning: uses joint or shared embedding space. The VLM is trained on datasets of image-text pairs and learns to minimize the distance between embeddings of matching pairs and maximize it for nonmatching pairs.
- ▶ Masking: models learn to predict randomly obscured parts of an input text or image.
- ▶ Generative model training: entails learning to generate new data
- ▶ Pretrained models: A pretrained LLM and a pretrained vision encoder can be used, with an added mapping network layer that aligns or projects the visual representation of an image to the LLM's input space.

VLM Architecture

- ▶ Qwen2.5 LM Decoder
- ▶ ViT:3D convolution to split input visual data into a series of 14*14 patches
- ▶ process video info: preprocessor for dynamic image size adjustment, dynamic frame extraction



Why this choice?

Ambiguity resolution

Symbolic planning in PDDL

A planning Problem is defined by:

- ▶ **Predicates:** Set of grounded states
- ▶ **Actions:** Set of possible moves, defined by preconditions and effects.
- ▶ **Planning Language $\mathcal{L}$:** the set of all possible predicates and variables.
- ▶ **Planning Domain $\mathcal{D} = (\mathcal{L}, A)$:** containing the set of usable predicates (states) and usable actions
- ▶ **Planning Problem $\mathcal{P} = (\mathcal{D}, I, G)$:** build with the domain and the set of initial states (or initial predicates) and the set of goal states (or goal predicates)

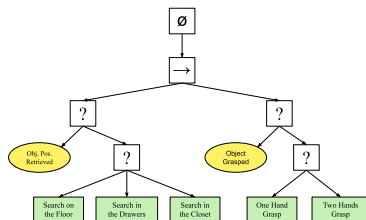
PDDL, stands for Planning Domain Definition Language, is the gold standards for Classical AI Planning. To solves The planning problem, two main approaches are possible: **State Space Search** and **Plan Space Search**.

To solve the Planning Problem **Fast Downward** has been used. It is an Heuristic Forward-Chaining State-Space Planner.

PDDL problem generation experiments

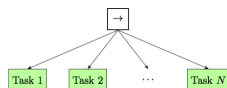
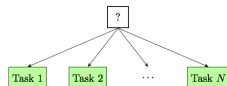
Executive Layer: Behavior Trees

- ▶ A **Behavior Tree** is a directed tree with nodes classified as root, control flow, or execution (tasks).
- ▶ Execution starts from the root, which sends *ticks* to its children at a set frequency.
- ▶ A node returns one of three statuses: **Running**, **Success**, or **Failure**.



Control Flow Nodes

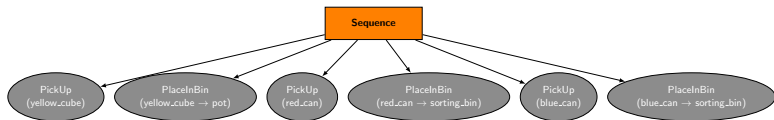
- ▶ **Selector:** A fallback node that executes the first child that does not fail. Returns **Success** or **Running**.
- ▶ **Sequence:** Executes children in order until one succeeds. Returns **Failure** or **Running** immediately if a child fails or is still running.



How to build Behavior Trees: `py_trees` library

`py_trees` is a Python library used to construct Behavior Trees efficiently. It relies primarily on two base classes: **Behaviours** and **Composites**.

- ▶ **Behaviour (Leaf):** The smallest element in a tree. Behaviours typically represent a **condition** (e.g., *Am I hungry?*) or an **action** (e.g., *Buy chocolate cookies*).
- ▶ **Composites (Control Flow):** Responsible for directing the path traced through the tree during a tick.
 - ▶ **Selectors:** Fallback decision-makers.
 - ▶ **Sequences:** Order-dependent factories.
 - ▶ **Parallels:** Concurrent execution factories.



BT Builder: Parsing and Sequence Compilation

1. Parsing PDDL Plan

(`parse_pddl_action`) Cleans raw planner text into structured Python data:

- ▶ **Filters Clutter:** Discards comments starting with ; (e.g., ; cost = 2) and blank lines.
- ▶ **Strips Planner Syntax:** Removes step prefixes (e.g., 0:) and outer parentheses (...).
- ▶ **Extracts Tokens:** Normalizes action name to lowercase and collects arguments:

```
"0: (place yellow_cube pot)" → ("place",  
    ["yellow_cube", "pot"])
```

2. Plan to Sequence

(`pddl_plan_to_sequence`) Converts the parsed action list into an executable Behavior Tree Sequence:

- ▶ **Skill Dispatch:** Uses `SKILL_REGISTRY` to map actions to robot skills ("pick", "place").
- ▶ **Argument Binding:** Passes simulator environment and object targets (object, asset).

Compilation Pipeline Example

1. Planner Output (Fast Downward):

```
; plan generated by fast downward  
0: (pick yellow_cube)  
1: (place yellow_cube pot)  
; cost = 2
```

⇓ *parsed & compiled into BT*

2. Executable PyTrees Sequence:

```
[~] pddl_plan_sequence (memory=True)  
|-- PickUp(object='yellow_cube')  
'-- PlaceInBin(object='yellow_cube',  
    asset='pot')
```

Design Rationale

Cleanly decouples **symbolic task planning** (Fast Downward) from the **physical robot control** (Motion planning).

BT Builder: Goal Checking and Reactive Tree Generation

3. Goal-Guarded Wrapper

(wrap_with_goal_check) A plain sequence is open-loop. If an object drops, it fails blindly. We wrap it in a reactive fallback loop:

- ▶ **Pre-Check (GoalCheck_Pre)**: Verifies if goal is already met before moving. If yes → succeeds immediately (no redundant actions).
- ▶ **Selector (memory=False)**: Re-evaluates continuously on every tick for dynamic preemption.
- ▶ **Post-Check (GoalCheck_Post)**: Confirms physical completion after the sequence finishes.
- ▶ **Retry Loop (Retry)**: If the grasp slips and post-check fails, automatically retries the plan up to N times.

4. Main Entrypoint

(build_bt_from_pddl_plan) Single builder API providing two execution modes:

- ▶ **wrap_goal_check=False**: Raw Sequence (open-loop baseline).
- ▶ **wrap_goal_check=True**: Self-healing tree with retry loop.

Goal-Guarded Tree Architecture

```
[?] goal_guarded_root (Selector)
|-- [>] GoalCheck_Pre (already met?)
'-- [R] retry_until_goal (max=10)
    '-- [->] attempt (Sequence)
        |-- [->] pddl_plan_sequence
        |   |-- PickUp(yellow_cube)
        |   '-- PlaceInBin(yellow_cube->pot)
        '-- [>] GoalCheck_Post (verified?)
```

Failure Recovery Example (Cube Drop)

1. Robot picks up cube, but gripper slips mid-air.
2. Sequence finishes → GoalCheck.Post checks pot.
3. Cube is missing → returns FAILURE.
4. Retry catches failure → re-runs the plan → task completed successfully!

BT Nodes: Skills & Conditions

Action Skills (`skills.py`)

Subclasses of `py_trees.behaviour.Behaviour`:

- ▶ **Non-Blocking Stepping**: Returns `RUNNING` while the robot moves, and `SUCCESS` or `FAILURE` upon completion.
- ▶ **TaskSpaceRRT Integration**: Computes collision-free $SE(3)$ end-effector trajectories dynamically for the 7-DOF Panda arm.
- ▶ **Operational Primitives**:
 - ▶ `MotionPlanningPickUp`: Approach → Grasp → Lift.
 - ▶ `MotionPlanningPlaceInBin`: Transit → Lower → Release → Retreat.

Condition Nodes (`conditions.py`)

Dual verification paradigm:

- ▶ **PDDLGoalCheck (Symbolic Grounding)**:
 - ▶ Parses goal formulas from `problem.pddl`.
 - ▶ Evaluates simulator predicates deterministically (`is_in_bin`, `is_grasped`).
 - ▶ Fast, deterministic, and zero-latency.
- ▶ **VisualCheck (VLM Inspection)**:
 - ▶ Adopts Wake et al. self-prompting.
 - ▶ Queries VLM (`vlm.query_fn`) on camera frames for open-world semantic validation.

Reactive Execution & Runtime Tree

Runtime Execution Flow

1. **Initial Check:** GoalCheck_Pre runs. If goal holds, the tree returns SUCCESS immediately (no redundant actions).
2. **Execution:** On failure, the Fallback ticks `retry_until_goal` → `pddl_plan_sequence`.
3. **Closed-Loop Motion:** Each skill executes collision-free RRT trajectories at 20 Hz.
4. **Verification & Recovery:** GoalCheck_Post validates final state. If an object is dropped, the Retry decorator triggers re-execution.

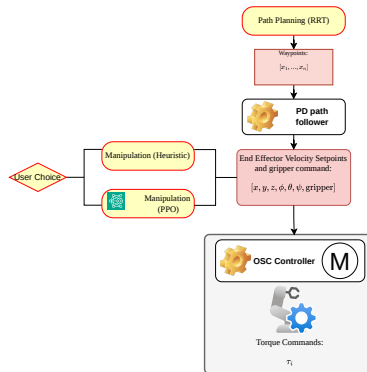
Generated Behavior Tree Structure

```
[?] goal_guarded_root (Selector)
|-- [>] GoalCheck_Pre
'-- [R] retry_until_goal (max=3)
    '-- [->] attempt (Sequence)
        |-- [->] pddl_plan_sequence
        |   |-- PickUp(yellow_cube)
        |   |-- PlaceInBin(yellow_cube->pot)
        |   |-- PickUp(red_can)
        |   '-- PlaceInBin(red_can->bin)
        '-- [>] GoalCheck_Post
```

Reactive Layer: Motion and Path Planning

Reactive Layer Roles

- ▶ **Symbolic-to-Continuous Grounding:** Translates discrete BT skills (PickUp, PlaceInBin) into continuous robot motion.
- ▶ **Perception:** Accounts for physical geometry, obstacles, and contact dynamics ignored by PDDL.
- ▶ **Motor Control:** Operates at 20 Hz with real-time state feedback from MuJoCo Simulator Environment.



Hierarchical Motion Decomposition

- ▶ **1. Path Planning (Task Space RRT):** Solves pure geometric obstacle avoidance in free space ($\mathbb{R}^3$).
- ▶ **2. Trajectory Tracking:** Closed-loop proportional tracking of Cartesian waypoints $[x_1, \dots, x_n]$ generated by the RRT.
- ▶ **3. Manipulation (PPO / Heuristic):** Governs fine contact dynamics, compliant grasping, and lifting.
- ▶ **4. Low Level Controller (OSC):** Takes as input the cartesian velocities from the Tracking and Manipulation modules and output the joint torques τ_i needed to control the robot.

Path Planning: RRT

Rapidly Random Trees (RRT) is a Probabilistic Path Planning method used to efficiently construct a path in continuous spaces.

Algorithm Goal Oriented RRT Algorithm

```
1:  $G.\text{init}(x_{\text{start}})$ 
2: for  $i = 1 \dots K$  do
3:    $p = \text{UniformRandom}(0, 1)$ 
4:   if  $p \leq p_{\text{goal}}$  then
5:      $x_{\text{rand}} \leftarrow x_{\text{goal}}$ 
6:   else
7:      $x_{\text{rand}} \leftarrow \text{RandPose}()$ 
8:   end if
9:    $x_{\text{nearest}} \leftarrow \text{NearestVertex}(x_{\text{rand}}, G)$ 
10:   $x_{\text{new}} \leftarrow \text{NewConf}(x_{\text{nearest}}, x_{\text{rand}}, \Delta x)$ 
11:  if  $\text{CollisionFree}(x_{\text{nearest}}, x_{\text{new}})$  then
12:     $G.\text{add\_vertex}(x_{\text{new}})$ 
13:     $G.\text{add\_edge}(x_{\text{nearest}}, x_{\text{new}})$ 
14:    if  $x_{\text{new}} \in X_{\text{goal}}$  then return Path Found
15:  end if
16: end if
17: end for
18: return  $G$ 
```

The goal oriented version samples the goal point with a given probability p while a random point with probability $1 - p$.

Trajectory Tracking

- ▶ **Cartesian Setpoints:** The waypoints generated by the task-space RRT are Cartesian poses used as setpoints for a **Proportional (P) Controller**.
- ▶ **Tracking Error:** At each time step, the position error is defined as:

$$\mathbf{e} = \mathbf{p}_{\text{target}} - \mathbf{p}_{\text{EF}}$$

- ▶ **Control Law:** The controller outputs velocity commands bounded by a maximum speed:

$$\mathbf{v}_{\text{cmd}} = \text{clip}(K_p \mathbf{e}, -v_{\text{max}}, v_{\text{max}})$$

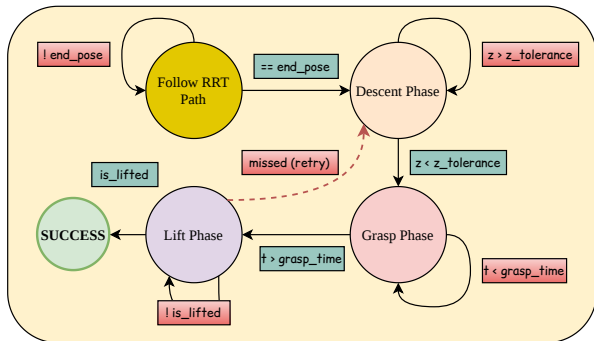
which are sent to the underlying operational space controller.

- ▶ **Application:** Used specifically during the **approach phase** in the pick action and during the **place action**. The place skill itself is completely based on the RRT planner and the trajectory tracker, while the pick skill is composed by more modules and need further explanation.

Manipulation: Heuristic Approach

Consists of a **Staged FSM**, that executes the following phases:

1. **RRT Planning:** Traverse collision-free waypoints to hover pose ($== \text{end_pose}$).
2. **Descent Phase:** Descend vertically until reaching grasp height ($z \leq z_{\text{tol}}$).
3. **Grasp Phase:** Actuate gripper firmly for set duration ($t > t_{\text{grasp}}$).
4. **Lift Phase:** Elevate object to safe height to verify pick. If it fails, it will retry from the Descent phase.



Proximal Policy Optimization

Proximal Policy Optimization is an on-policy actor-critic Reinforcement Learning method. PPO updates policy parameters θ_{k+1} by maximizing the **Clipped Surrogate Objective**:

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]$$

where the probability ratio is defined as:

$$r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_k}(a_t | s_t)}$$

- ▶ **Clipped Objective:** The hyperparameter $\epsilon \approx 0.2$ penalizes moving $r_t(\theta)$ outside $[1 - \epsilon, 1 + \epsilon]$, preventing destructively large policy updates.
- ▶ **Actor-Critic Architecture:** Jointly optimizes a policy network $\pi_{\theta}(a|s)$ and a state-value baseline $V_{\phi}(s)$ using Generalized Advantage Estimation (GAE).

Manipulation: PPO Approach

TaskSpaceRRT provides coarse global navigation to a pre-grasp hover pose (~ 18 cm above the object). The **PPO policy** then takes over using a minimal, translation-invariant **11D observation vector**:

$$\mathbf{o}_t = \left[\underbrace{\Delta \mathbf{p}}_{\mathbb{R}^3}, \underbrace{\mathbf{q}_{\text{grip}}}_{\mathbb{R}^2}, \underbrace{\dot{\mathbf{q}}_{\text{grip}}}_{\mathbb{R}^2}, \underbrace{z_{\text{rel}}^{\text{EEF}}}_{\mathbb{R}^1}, \underbrace{\Delta z_{\text{obj}}}_{\mathbb{R}^1}, \underbrace{c_{\text{grasp}}}_{\{0,1\}}, \underbrace{m_{\text{geom}}}_{\{0,1\}} \right]^T \in \mathbb{R}^{11}$$

Kinematics & Spatial Relations

- ▶ $\Delta \mathbf{p} = \mathbf{p}_{\text{obj}} - \mathbf{p}_{\text{EEF}}$: 3D relative vector (translation invariance across table).
- ▶ $z_{\text{rel}}^{\text{EEF}} = p_{\text{EEF},z} - z_{\text{table}}$: Table clearance to avoid surface collisions.
- ▶ $\Delta z_{\text{obj}} = p_{\text{obj},z} - p_{\text{obj},z}^{(0)}$: Vertical displacement measuring lift progress.

Gripper State & Morphology

- ▶ $\mathbf{q}_{\text{grip}}, \dot{\mathbf{q}}_{\text{grip}}$: Left/right finger joint positions and velocities (contact stall detection).
- ▶ $c_{\text{grasp}} \in \{0, 1\}$: Bilateral force-closure flag (Robosuite contact normal check).
- ▶ $m_{\text{geom}} \in \{0, 1\}$: Shape token (0: cube, 1: can) enabling multi-geometry conditioning.

Manipulation: Shaped Reward Formulation

The agent is trained with a multi-stage continuous-discrete reward guiding the transition from approach $\rightarrow$ grasp $\rightarrow$ lift:

$$R_t = r_{\text{reach}} + r_{\text{grip}} + r_{\text{grasp}} + r_{\text{lift}} - c_{\text{step}} + r_{\text{terminal}}$$

Continuous Shaping Terms

- **Smooth Reaching** ($d = \|\Delta \mathbf{p}\|_2$):
 $r_{\text{reach}} = 1.0 - \tanh(10.0 \cdot d)$
- **Grasp Maintenance:**
 $r_{\text{grasp}} = 2.0 \cdot \mathbb{I}(\text{grasped})$
- **Conditioned Vertical Lift:**
 $r_{\text{lift}} = 20.0 \cdot \max(0, \Delta z_{\text{obj}}) \cdot \mathbb{I}(\text{grasped})$
(Avoids lifting with an empty gripper)
- **Alive Cost:** $c_{\text{step}} = 0.01$ (incentivizes speed)

Action Shaping & Terminals

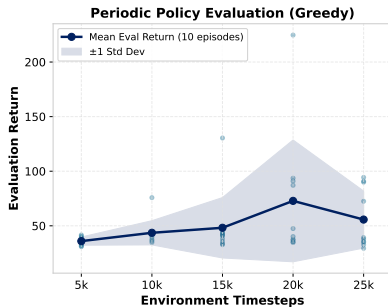
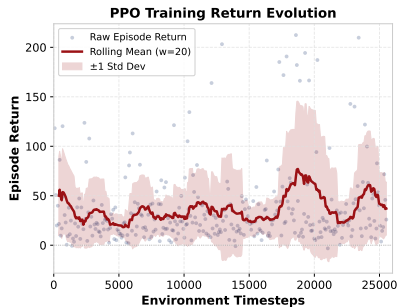
- **Gripper Timing Incentive** r_{grip} :

$$r_{\text{grip}} = \begin{cases} +0.5 & \text{if } d < 4 \text{ cm} \wedge a_g > 0 \\ -0.25 & \text{if } d < 4 \text{ cm} \wedge a_g \leq 0 \\ -0.2 & \text{if } d \geq 4 \text{ cm} \wedge a_g > 0.2 \\ 0 & \text{otherwise} \end{cases}$$

Prevents premature closure before alignment.

- **Terminal Sparse Rewards:**
 $+15.0$ if $\Delta z_{\text{obj}} \geq 6 \text{ cm} \wedge \text{grasped}$
(Success)
 -2.0 if $p_{\text{obj},z} < z_{\text{table}} - 4 \text{ cm}$ (Drop)

Manipulation: PPO Training & Evaluation



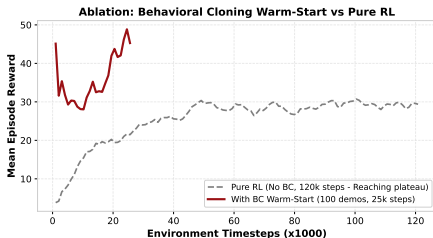
Training Progression (25k Steps)

- **Exploration Phase:** High variance in initial rollouts as the policy explores grasp stability.
- **High-Return Peaks (> 150):** Successful grasp + vertical lift + sparse completion bonus (+15).

Deterministic Evaluation

- Evaluated every 5,000 steps over 10 test rollouts.
- Greedy policy achieves consistent ascent, climbing from initial hover (~ 36) to peak performance (~ 73).

Manipulation: Why Behavioral Cloning Warm-Start?



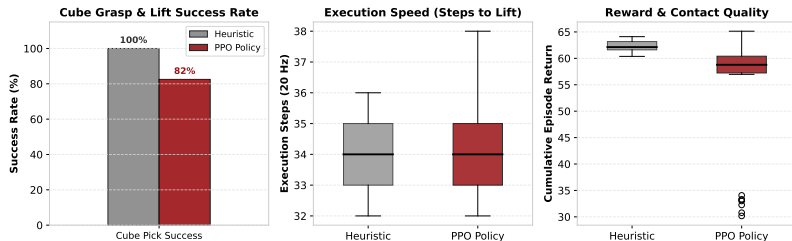
Algorithmic Oracle Demonstrations

- ▶ Generated via a closed-loop 3-stage state-machine using simulation ground-truth poses.
- ▶ 100 demos ($\sim 3,000$ transitions) pre-trained for 150 epochs via supervised MSE loss.

Key Empirical Findings

- ▶ **Pure RL (120k steps):** Gets stuck in the "reaching local minimum" (≈ 29 reward), never discovering the precise descent-grasp-lift sequence.
- ▶ **BC Warm-Start (25k steps):** Initializes policy in the basin of attraction, achieving immediate task competence in $5\times$ fewer steps.

Manipulation: Heuristic vs PPO Comparison



Cube Manipulation Performance

- ▶ **High Reliability:** PPO achieves **82.5%** success rate (vs. 100% analytical oracle) using local relative observations.
- ▶ **Micro-Trajectory Parity:** Identical physical motion speed: **1.71 s** (34.1 steps) vs **1.70 s** (34.0 steps).
- ▶ **Eliminates Dwell Timers:** The heuristic forces a conservative **2.0 s** (40 ticks) pause before lifting; PPO lifts instantly upon contact.

Trade-Offs & Hybrid Synergy

- ▶ **Dynamic Compliance:** PPO modulates continuous velocities based on real-time force closure rather than rigid fixed gains ($K_p = 10$).
- ▶ **Privileged vs Reactive:** Heuristic requires exact object coordinates; PPO relies only on local $\Delta \mathbf{p}$ and joint kinematics.
- ▶ **Safety Guard:** Hybrid pick node falls back to heuristic if step budget (≤ 90) expires.

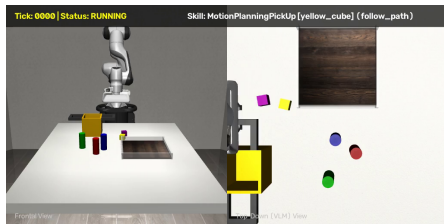
Experimental Setup: Simulation Environment

Robosuite TaskSorting Arena

- ▶ **Robot:** 7-DoF Franka Emika Panda with parallel-jaw gripper (20 Hz OSC control).
- ▶ **Physics:** MuJoCo headless physics engine with compliant contact dynamics.

Sensory & Tabletop Assets

- ▶ **Dual Viewpoints:** Orthographic *top-down* (VLM grounding) + perspective *frontview* (monitoring).
- ▶ **Objects:** 5 pickable items (yellow/purple cubes, red/blue/green cans).
- ▶ **Containers:** Sorting bin (30×30 cm) + handled pot (collision obstacles).



End-to-End TAMP Pipeline Execution

Task Mission: *"Put the red can in the sorting bin, then put the yellow cube in the pot, then put the blue can in the sorting bin."*

1. Perception & Plan

- ▶ **Qwen 2.5-VL:** Resolves object locations from top-down frame.
- ▶ **Llama 3:** Synthesizes initial/goal PDDL predicates.
- ▶ **Fast Downward:** Generates 6-step symbolic action plan in < 0.1 s.

2. BT Compilation

- ▶ Compiled into reactive `py_trees` structure.
- ▶ Encapsulates pre-conditions, action sequences, and fallback recovery nodes.
- ▶ Continuous state tick monitoring at 20 Hz.

3. Motion & Control

- ▶ **TaskSpaceRRT:** Computes collision-free transit paths (~ 30 ms).
- ▶ **Hybrid PPO:** Compliant grasp and lift (≈ 1.7 s).
- ▶ **Placement:** Waypoint tracking to bin/pot.

Experimental Results: Subsystem Performance

Quantitative evaluation across end-to-end multi-object sorting missions:

Pipeline Layer	Module / Algorithm	Mean Latency	Success Rate
Visual Grounding	Qwen 2.5-VL (3B)	1.25 s	100%
PDDL Formulation	Llama 3 (8B)	1.80 s	100%
Symbolic Planning	Fast Downward (A^*)	0.07 s	100%
Motion Planning	TaskSpaceRRT	0.03 s	100%
Manipulation (Pick)	Hybrid PPO Policy	1.71 s	88% (PPO)
Fallback Execution	Deterministic Staged	1.70 s	100% (Fallback)
End-to-End System	Complete Architecture	Total Mission	100%

Key Findings

- ▶ Symbolic & motion planning combined require < 0.1 s, enabling agile replanning.
- ▶ Zero collisions observed with container walls across all transit trajectories.

Execution Guarantees

- ▶ The hybrid RL-fallback mechanism ensures zero mission halts on manipulation failures.
- ▶ Modular design decouples high-level task planning from local contact mechanics.